

# 다 됐나요?

은행 앱이 말해주지 않는 실패 원인을, 근거와 함께 판정합니다

팀명 다 됐나요 · 2인

제8회 KB Future Finance A.I. Challenge · 예선 기술설명서

데모 — <https://kb-ai-challenge-2026.vercel.app>

2026-07-29

# 한 장 요약

**문제 — 은행 앱은 "안 됩니다" 라고만 말하고, 무엇이 막고 있는지는 말하지 않습니다.**

**해결** — KB 공개문서에서 조건을 **자동 추출**해 기계가 판정할 수 있는 **조건 트리**로 만들고, 사용자 상태와 대조해 **막힌 항목 + 근거 원문 + 다음 행동**을 지목합니다.

KB 공개문서 —[LLM]→ 조건 트리 —[규칙엔진]→ 무엇에 막혔는가  
 약관·FAQ 추출 재사용 자산 + 근거 원문  
 8건 조건 12개 + 앱에서 되는지  
 + 하나만 풀면 되는지

| 지금 상태                              |                                                      |
|------------------------------------|------------------------------------------------------|
| 문서 8건 → 청크 280개 → 조건 12개           | 전부 자동 추출 · 검수 통과                                     |
| 근거가 원문에 실재                         | 12 / 12                                              |
| 판정 2,000회 반복 일치율                   | 100%                                                 |
| 판정 1회 소요 (중앙값)                     | 0.056 ms                                             |
| 직접 재현 — <code>pytest tests/</code> | 32 passed (결정론 1 · 엣지 6 · 상태변경 8 · 근거묶음 10 · 적용범위 7) |

# 1. 문제 정의

## 앱은 실패를 알려주지만, 원인은 알려주지 않는다

앱: "신분증이 인식되지 않습니다. 다시 촬영해 주세요."

사용자: 재촬영 반복 → 포기

실제: 비대면 계좌개설 안심차단이 켜져 있음

→ 다시 찍어도, 다른 은행에 가도 개설되지 않음

→ 해제는 영업점에서만 가능

"확인하세요"와 "확인해드렸습니다"의 차이. 검색하면 FAQ가 나옵니다. 그러나 FAQ는 *확인하라고* 말할 뿐, **지금 내 상태가 어떤지는 확인해주지 않습니다.**

이 프로젝트는 팀장이 직접 겪은 일에서 시작했습니다 — *"해외결제를 풀려고 하나 풀었는데 안 되고, 하나 더 풀 게 있더라. 어디 있는지 몰라 검색했다."*

# 1. 문제 정의 (계속)

앱은 실패를 알려주지만, 원인은 알려주지 않는다 (계속)

저희가 서는 자리 — 실패한 직후, 다시 시도하기 전

| 가입·신청 전                  | 실행 시도           | 실패 직후                    | 며칠 뒤                 |
|--------------------------|-----------------|--------------------------|----------------------|
| 상품 추천 · 자격 진단<br>(사전 예방) | 거래 실행<br>(에이전트) | ★ 여기가 비어 있다<br>"왜 안 됐지?" | 민원 · 분쟁조정<br>(사후 구제) |

사전 예방은 아직 시도하지 않은 사람을 돕고, 사후 구제는 이미 손해가 난 사람을 돕습니다. 그 사이, **방금 실패해서 다시 시도하려는 사람**에게는 아무것도 없습니다. 앱은 실패만 알리고 원인을 말하지 않으며, 검색은 일반론을 줍니다.

그 몇 분이 이 시스템의 자리입니다. 이 순간의 사용자는 **판단할 재료**만 있으면 스스로 해결합니다 — 무엇이 막고 있는지, 그 근거가 어디에 있는지.

## 2. 문제의 크기 — 추정이 아니라 실측

조건은 흩어져 있고, 화면은 일부만 보여준다

|                        | 값     | 출처                       |
|------------------------|-------|--------------------------|
| KB Pay 설정 화면에 보이는 항목   | 2개    | 실측 (2026-07-27)          |
| 공개문서에서 자동 추출한 조건       | 11개   | data/trees/              |
| 그중 앱 안에서 해결할 수 있는 조건   | 2개    | remedy.actionable_in_app |
| 앱 밖에서 해결해야 하는 조건       | 3개    | 근거 원문에 채널이 적힌 것만         |
| 해결 경로를 수집 문서에서 못 찾은 조건 | 6개    | 추측하지 않고 비워 둡니다           |
| 사람이 조건 7개를 직접 확인한 시간   | 326초  | 스톱워치 실측, 측정자 2인          |
| 그중 검색을 거친 조건           | 4 / 7 | 앱 메뉴로 도달한 사례 0건          |
| 결론에 도달하지 못한 조건         | 1건    | FAQ가 "고객센터 문의"로 종결       |
| 인증 장벽으로 측정조차 못한 조건     | 2건    | 카드 비밀번호 미기억              |

326초는 **찾아야 할 목록을 미리 알고 켜 시간**입니다. 무엇을 확인해야 하는지 모르는 실제 사용자는 더 오래 걸립니다 — 이 수치는 **하한**입니다. (7개는 측정 시점 기준이며, 자동 추출본은 11개입니다)

# 3. 접근 — 생성이 아니라 검증

## Retrieval-Augmented Verification

일반적인 RAG는 검색 결과를 **문장으로 요약**합니다. 그러면 답이 매번 달라지고, 틀려도 그럴듯합니다.

저희는 검색 결과를 **실행 가능한 술어(predicate)**로 바꿉니다.

|         |                      |                 |
|---------|----------------------|-----------------|
| RAG     | 문서 → LLM → 자연어 답변    | (매번 다름 · 검증 불가) |
| RAV(우리) | 문서 → LLM → 조건 트리(구조) | ← 여기까지만 LLM     |
|         | ↓                    |                 |
|         | 사용자 상태 → 규칙 엔진 → 판정  | ← 여기부터는 결정론     |

|     | LLM 이 하는 일                 | LLM 이 하지 않는 일             |
|-----|----------------------------|---------------------------|
| 범위  | 비정형 문서에서 조건을 <b>뽑아 형식화</b> | <b>판정 · 해결 계획 · 사유 설명</b> |
| 결과물 | 조건 트리 (파일로 고정)             | —                         |
| 재현성 | 캐시 + temperature=0         | 같은 입력 = 같은 출력 (측정)        |

**미충족 사유를 문장으로 생성하지 않습니다.** 근거 원문을 그대로 보여줍니다. 설명을 생성하면 원문 밖 문장이 나올 위험이 생깁니다 — 그 위험을 **아예 만** 들지 않는 쪽을 택했습니다.

**이 분리가 이 제안의 핵심입니다.** 금융 판정은 틀리면 사용자를 잘못된 행동으로 보냅니다. 그래서 판정에서 LLM을 뺐고, 뺐다는 사실을 **숫자로 증명**합니다.

### 3. 접근 — 생성이 아니라 검증 (계속)

#### Retrieval-Augmented Verification (계속)

#### 왜 에이전트로 만들지 않았는가

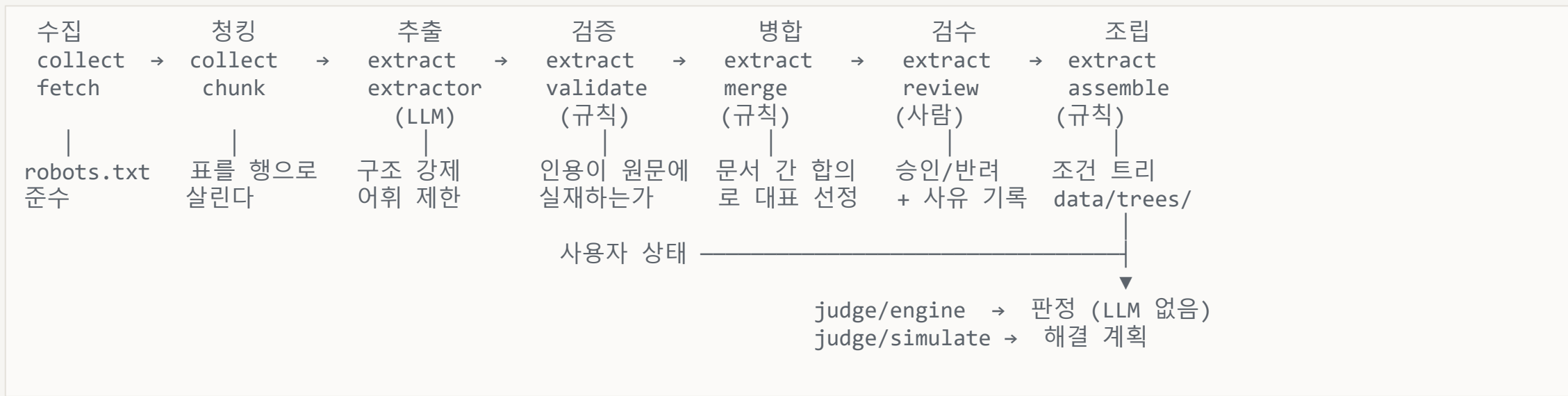
에이전트는 대신 해줍니다. 저희는 판정만 합니다. 의도한 선택입니다.

|        | 에이전트                  | 이 시스템                        |
|--------|-----------------------|------------------------------|
| 하는 일   | 사용자를 대신해 실행           | 무엇이 막고 있는지 <b>지목</b>         |
| 틀렸을 때  | 잘못된 실행이 <b>이미 일어남</b> | 사용자가 근거 원문을 보고 <b>직접 판단</b>  |
| 필요한 권한 | 계좌·카드 제어 권한           | <b>읽기만</b>                   |
| 검증     | 실행 로그를 사후에 본다         | 조건·상태·판정이 <b>화면에 동시에</b> 보인다 |

해외거래정지를 대신 해제해주는 편이 더 친절해 보입니다. 그러나 그것이 틀렸을 때 사용자의 카드는 이미 열려 있습니다. **저희는 아무것도 실행하지 않습니다.** 막힌 조건과 그 근거를 보여주고, 행동은 사용자가 합니다.

그래서 이 시스템은 **틀려도 사용자가 알아챌 수 있습니다.** 모든 판정에 근거 원문·출처·수집일이 붙어 있고, 화면에서 값을 바꿔 판정이 어떻게 달라지는지 직접 확인할 수 있기 때문입니다.

## 4. 시스템 구조



LLM 은 extract/extractor 한 곳에만 있습니다. 나머지는 전부 규칙과 사람입니다.



# 5. 핵심 산출물 — 조건 트리 (C1)

이 프로젝트가 남기는 자산은 서비스가 아니라 데이터입니다

```
{
  "id": "c_nonface_open_block",
  "label": "비대면 계좌개설 안심차단이 해제되어 있어야 합니다",
  "predicate": { "subject": "account.nonface_open_block", "op": "eq", "value": false },
  "severity": "blocking",
  "remedy": { "actionable_in_app": false, "channels": ["branch:영업점"],
    "primary_path": "영업점 방문 (신분증 지참)",
    "basis": "evidence" },
  "evidence": {
    "source_title": "비대면 계좌개설 안심차단으로 보이스피싱 피해 예방하기",
    "url": "https://kbthink.com/main/living-finance/talk-cardnews/2025/...",
    "quote": "... 서비스 해제는 반드시 지점을 방문해 직접 신청해야 합니다. ...",
    "collected_at": "2026-07-27", "confidence": "high"
  },
  "provenance": {
    "support_count": 5,
    "source_ids": ["kbthink_safeblock", "obank_nonface_safeblock"]
  }
}
```

## 5. 핵심 산출물 — 조건 트리 (C1) (계속)

### 이 프로젝트가 남기는 자산은 서비스가 아니라 데이터입니다 (계속)

(실제 `data/trees/account_open_nonface.json` 의 조건을 인용문만 줄여 옮긴 것입니다)

- **evidence** 는 스키마상 필수입니다 → 근거 없는 조건은 존재할 수 없습니다
- **actionable\_in\_app** 은 세 값입니다. 근거로 확인되지 않으면 null 이며, 무엇이 그 판단을 뒷받침하는지는 **basis** 에 남깁니다 (7-1 참조)
- **provenance** 는 문서 몇 곳 · 문장 몇 개에서 확인됐는지를 남깁니다. **support\_count** 는 문장(청크) 수이고 문서 수는 **source\_ids** 로 셉니다 — 위 예시는 문서 2곳 · 문장 5개입니다. 화면도 그렇게 표시합니다
- 화면에는 「조건 원본」 절이 따로 있습니다. 판정이 「가능」이어도 이 절은 그대로 남습니다 — 자산은 판정에 종속되지 않기 때문입니다. 화면 첫머리와 절 경계에서 “여기부터는 판정과 별개입니다” 라고 범위를 명시적으로 끊습니다

# 6. 추출 — 통제된 어휘로 형식화한다

## directed symbolic prompting

LLM에게 자유롭게 조건을 만들게 하면 존재하지 않는 상태 경로를 지어냅니다. 그 조건은 사용자 프로필과 매칭되지 않아 영원히 unknown 이 됩니다.

그래서 참조 가능한 어휘를 미리 못 박습니다.

|                    | 수  | 예                                                         |
|--------------------|----|-----------------------------------------------------------|
| 허용 subject (상태 경로) | 21 | card.overseas_block_online,<br>account.nonface_open_block |
| 허용 op (연산자)        | 16 | eq, lte, date_after, count_lte                            |
| 허용 category        | 5  | setting / document / limit / temporal / eligibility       |

근거: *Neuro-Symbolic Framework for Public-Sector AI* (arXiv:2512.12109, FAccT 2026) — 통제 어휘를 명시하면 형식화 성공률이 크게 오른다.

목록에 맞는 경로가 없으면 그 조건은 추출하지 않습니다. 빠뜨리는 비용이 틀리는 비용보다 작기 때문입니다.

# 7. 검증 게이트 — "근거 100%"를 구조로 보증한다

## 말이 아니라 코드가 막는다

| # | 게이트           | 막는 것                              |
|---|---------------|-----------------------------------|
| 1 | 인용 실재 검증      | 모델이 지어낸 인용 — 원문에 없으면 폐기           |
| 2 | subject 허용목록  | 존재하지 않는 상태 경로                     |
| 3 | op 허용목록       | 평가할 수 없는 연산자                      |
| 4 | value JSON 파싱 | 빈 값·형식 오류                         |
| 5 | confidence 규칙 | medium/low 인데 사유가 없는 조건           |
| 6 | 해결 경로 근거 대조   | 원문이 채널을 말하지 않는데 붙은 해결 방법 (7-1 참조) |

## 인용 자동 보정

모델은 원문 문장 앞뒤에 **없는 말을 덧붙이는** 실수를 반복했습니다. 그대로 폐기하면 **실재하는 조건이 사라집니다**.

|    |                                               |
|----|-----------------------------------------------|
| 원문 | "... 권유 드립니다. 원화 결제 시 추가로 부과되는 ..."           |
| 모델 | "해외 가맹점에서 원화 결제 시 추가로 부과되는 ..." ← 앞에 없는 구절    |
| 보정 | 원문에 실제로 있는 최장 연속 구간만 남긴다 → 결과는 언제나 원문의 부분 문자열 |

LLM 에게 사유를 알려주고 다시 묻는 방법을 **먼저 시도했습니다** — **3건 중 0건 복구**. 같은 실수를 반복했습니다. 그래서 규칙으로 바꿨고, 1건을 되살렸습니다.

# 7-1. 게이트가 놓친 것 — 해결 방법에는 근거가 없었습니다

## 인용은 검증했는데, 해결 방법은 검증하지 않았습니다

제출 직전 화면에서 이상한 것이 보였습니다. 같은 성격의 한도 조건 셋 중 하나만 「앱에서 가능」이었습니다.

|                      |        |                    |
|----------------------|--------|--------------------|
| 1일 이용한도 이내여야 합니다     | 앱에서 가능 | · "한도 조정" (KB Pay) |
| 1회 이용한도 이내여야 합니다     | 앱에서 불가 |                    |
| 누적 이용액이 ... 이내여야 합니다 | 앱에서 불가 |                    |

근거 원문을 열어 보니 **1일 한도 | 현금인출/가맹점이용 | 600만원 상당의 달러(USD) 환산액** — 한도 수치만 있고 조정 방법은 한 글자도 없었습니다. "한도 조정"이라는 말은 KB 문서 어디에도 없었고, **저희 프롬프트 예시에만** 있었습니다.

## 원인 두 가지

|      | 무엇이 잘못됐나                                                                          |
|------|-----------------------------------------------------------------------------------|
| 스키마  | actionable_in_app: bool — "모른다"를 표현할 자리가 없어 모델이 반드시 참/거짓 중 하나를 고르게 <b>강요</b> 했습니다 |
| 프롬프트 | few-shot 예시에 "primary_path": "한도 조정" 이 있었고, 한도 조건을 만나자 그대로 베꼈습니다                  |

조건·상태·판정에서는 「모르면 모른다」를 지켰으면서 **remedy**에서만 그 원칙이 빠져 있었습니다. 검수자도 조건과 인용만 봤지 해결 방법은 보지 않았습니다.

# 7-1. 게이트가 놓친 것 — 해결 방법에는 근거가 없었습니다 (계속)

인용은 검증했는데, 해결 방법은 검증하지 않았습니다 (계속)

고친 방법 — 네 겹으로

| # | 조치                                                                                                      |                                     |
|---|---------------------------------------------------------------------------------------------------------|-------------------------------------|
| 1 | 스키마를 세 값으로. `actionable_in_app: bool`                                                                   | None. `**false` 도 주장**이므로 근거가 필요합니다 |
| 2 | remedy.basis 신설 — evidence(원문에 채널 명시) / measured(앱에서 직접 확인) / self_evident(물리적으로 자명). 셋 중 어디에도 없으면 비웁니다 |                                     |
| 3 | 검증 게이트에 6번 관문 추가. 원문에 채널을 가리키는 말이 없으면 remedy 를 자동으로 비웁니다                                                |                                     |
| 4 | 프롬프트 예시를 정반대로 교체. 이제 그 예시가 null 을 가르칩니다                                                                 |                                     |

# 7-1. 게이트가 놓친 것 — 해결 방법에는 근거가 없었습니다 (계속)

인용은 검증했는데, 해결 방법은 검증하지 않았습니다 (계속)  
이 구분은 화면에도 그대로 나옵니다

조건마다 무엇이 그 해결 방법을 뒷받침하는지를 칩으로 답니다.

| 칩             | basis                                | 뜻                           |
|---------------|--------------------------------------|-----------------------------|
| 근거 원문에 채널 명시  | evidence                             | 원문이 <b>어디서</b> 하는지까지 말했습니다  |
| 근거 원문에 할 일 명시 | evidence (+ actionable_in_app: null) | 원문이 <b>무엇을</b> 하라고만 말했습니다   |
| 앱에서 직접 확인     | measured                             | 문서에 없어 <b>저희가 앱에서 재봤습니다</b> |
| 조건 자체로 자명     | self_evident                         | 조건 문장이 곧 해결 방법입니다           |

칩이 없는 조건은 **해결 방법을 찾지 못한 조건**입니다. 비워 둔 것도 화면이 말합니다.

# 결과

전 조건 11개 중 8개 "앱에서 해결 불가" ← 그중 5개는 근거가 없었음  
후 앱에서 가능 2 · 앱 밖 3 · 경로 미확인 6

숫자는 줄었지만 주장은 정확해졌습니다. 그리고 문제 제기는 오히려 날카로워집니다 — *앱이 말해주지 않는다*가 아니라, **문서조차 말해주지 않는 조건이 6개**라는 뜻이기 때문입니다.

tests/test\_remedy\_grounding.py 10건이 이 원칙을 지킵니다. 근거 없는 remedy 를 되돌려 넣어 보면 테스트가 실제로 실패하는 것을 확인했습니다.

## 한 번 더 확인했더니, 이번엔 반대 방향의 오류가 있었습니다

비운 6개를 수집 문서 8건의 본문 전체에서 다시 검색했습니다 (HTML 파일 기준 82만 자, 태그·스크립트를 걷어낸 본문 기준 11만 3천 자). 그중 하나는 근거가 있었습니다.



## 결과 (계속)

### 그리고 "KB 공개문서에 없다"고 말하지 않습니다

카드 유효기한을 확인해주세요. 해외에서 카드를 사용해야하는 기간 동안 만료가 예상되는 경우 사용 전 갱신 재발급이 필요합니다.

**무엇을 해야 하는지는 원문에 있습니다** — 「갱신 재발급」. 다만 그 재발급을 **어디서** 하는지는 없습니다. 6개를 한 덩어리로 처리하면서 이 차이를 놓쳤습니다. 되살리되 두 층위를 구분했습니다.

|         | 할 일(primary_path) | 채널(actionable_in_app) |
|---------|-------------------|-----------------------|
| 카드 유효기한 | 갱신 재발급 (원문에 있음)   | 미확인 (원문에 없음)          |
| 나머지 5개  | 없음                | 미확인                   |

저희가 본 것은 **수집한 문서 8건**입니다. KB 전체가 아닙니다. 그래서 화면과 문서의 문구를 "**수집한 문서에서 찾지 못했습니다**"로 좁혔습니다.

다른 페이지에 해결 방법이 있을 수 있습니다. 그 경우 **수집 범위를 넓히면 자동으로 채워집니다** — 파이프라인이 문서를 넣으면 조건 트리를 내는 구조이기 때문입니다. 지금 비어 있다는 사실 자체가 "이 안내들만 보고는 알 수 없다"는 뜻이고, 그것이 이 프로젝트가 말하려는 문제입니다.

## 8. 프롬프트 규칙 — 추측이 아니라 관측

### 1차 추출을 분석해 실패 7종을 규칙으로 못 박았다

| # | 관측된 실패                             | 규칙                                        |
|---|------------------------------------|-------------------------------------------|
| ① | 방향 뒤집힘 — "안심차단이 커져 있어야 한다"         | <i>이 predicate 가 참이면 목표가 달성되는가?</i>       |
| ② | subject 오매핑 — 서로 다른 조건을 한 경로에 몰아넣음 | 뜻이 정확히 맞지 않으면 추출하지 않는다                    |
| ③ | 목표와 무관 — 이의신청 기한, 수수료 면제           | 지금 그 목표를 달성하는 조건만                         |
| ④ | 단위 혼동 — "5,500달러" → 5500000        | 원화 기준이 아니면 추출하지 않는다                       |
| ⑤ | 인용 이어붙임                            | 연속된 한 구간만                                 |
| ⑥ | value 를 비움                         | 값이 없어도 "null" 을 적는다                       |
| ⑦ | label 과 predicate 방향 불일치           | <i>label 을 predicate 로 다시 쓰면 같은 문장인가?</i> |

1차 결과는 병합 15개 중 8개가 위 유형으로 틀렸습니다. 규칙을 넣고 5차까지 개정한 결과가 지금의 트리입니다.

이 규칙들은 실제로 나온 오류입니다. 지울 때는 그 실패가 재현되는지 먼저 확인하도록 코드 주석에 남겼습니다.

## 9. 병합 — 문서 간 합의로 대표를 정한다

---

### 사람이 고친 자리는 화면에도 남습니다

검수에서 사람이 판단을 뒤집으면 `review_override` 에 **이유까지** 남고, 그 문장이 **화면의** 근거 원문 아래에 그대로 나옵니다. 감사 기록을 파일에만 두면 아무도 읽지 않습니다 — 조건을 보는 그 자리에 둡니다.

# 길이는 신뢰의 근거가 아니다

같은 조건이 여러 문서에 조금씩 다른 문장으로 실립니다. 무엇을 보여줄지 정해야 합니다.

- X 짧은 인용 우선 → 규칙을 스치듯 언급한 문장이 대표가 됨
- X 긴 인용 우선 → 다른 얘기를 하는 긴 문단이 대표가 됨  
("해외거래정지 해제"(문서 5곳)가 "해외사이트 해제신청"(1곳)에 밀림)
- o 문서 간 합의 → 같은 뜻으로 읽은 문서가 가장 많은 무리를 대표로  
그 안에서 (여러 문서에 같은 문장 → label 뒷받침 → 짧은 것) 순

## 합집합을 쓰지 않는 이유

해결 채널을 문서들의 합집합으로 모았더니, "앱에서 안심차단을 신청할 수 있어요"의 app:KB스타뱅킹 이 **해제** 조건에 섞여 "앱에서 해제 가능"으로 뒤집혔습니다.

→ 화면에 보이는 모든 정보는 하나의 인용에서 나옵니다. 문서마다 값이 다르면 합치지 않고 검수 메모로 사람에게 넘깁니다.

# 10. 검수 관문 — 사람은 게이트지 저자가 아니다

프롬프트를 고쳐도 오류가 0이 되지는 않습니다. 틀린 조건이 트리에 들어가면 판정은 자신 있게 틀린 답을 냅니다. 그게 가장 나쁩니다.

| 사람이 할 수 있는 것                         | 사람이 할 수 없는 것    |
|--------------------------------------|-----------------|
| 승인 / 반려                              | label 을 쓰거나 고치기 |
| severity 교정 (blocking ↔ warning)     | predicate 수정    |
| 여러 인용 중 <b>대표 선택</b> (모델이 뽑은 것 중에서만) | evidence 문장 작성  |

검수 대상 16건 → 승인 12 / 반려 4 (통과율 0.75)  
severity 교정 2 · 대표 인용 선택 1

- 결정과 **사유 전문**이 data/review/decisions.yaml 에 남습니다 — 감사 기록입니다
- 교정한 조건은 트리에 review\_override 로 표시돼 **손댄 곳이 감춰지지 않습니다**
- 검수되지 않은 조건은 **트리에 들어가지 않습니다** (모르면 넣지 않는다)

# 11. 판정 엔진 — 결정론

같은 입력에 같은 출력. 측정했습니다.

| 항목             | 값        |
|----------------|----------|
| 조합 (트리 × 프로필)  | 10       |
| 총 판정 횟수        | 2,000    |
| 일치율            | 100.0%   |
| 판정 1회 소요 (중앙값) | 0.056 ms |

비교 시 실행 시각을 제외하고 조건 목록을 id 로 정렬해 대조했습니다. 판정과 해결 계획을 **함께** 직렬화해 비교했으므로, 둘 중 하나만 흔들려도 불일치로 잡힙니다.

조합에는 **목표와 무관한 프로필**(계좌 트리 × 해외결제 사용자)도 넣었습니다. 이 경우 시스템은 indeterminate 를 냅니다 — 관련 정보가 없으면 추측하지 않습니다.

# 11-2. 그 결정론을 테스트로 고정했습니다

측정치를 주장으로만 두지 않았습니다. 저장소를 받은 사람이 직접 다시 돌릴 수 있습니다.

| 스위트                                  | 지키는 것                                             | 실패하면                                         |
|--------------------------------------|---------------------------------------------------|----------------------------------------------|
| tests/test_determinism.py            | 트리 × 프로필 전체 조합(10)을 각 10회 반복해 판정이 모두 일치           | 판정 경로에 <b>LLM 호출</b> 이나 <b>난수가 끼어들었다</b> 는 뜻 |
| tests/test_edge.py (6건)              | 빈 프로필 · null · 없는 goal_id · 타입 불일치 · 빈 트리         | <b>모름을 충족으로 추측</b> 했거나 500을 냈다는 뜻            |
| tests/test_overrides.py (8건)         | 화면에서 바꾼 값이 판정에 반영되고, <b>원본 프로필은 그대로</b> 인가        | 값에 반응하지 않거나, 다음 사람이 보는 화면이 오염됐다는 뜻           |
| tests/test_remedy_grounding.py (10건) | 해결 방법이 <b>근거에 묶여 있는가</b> . false 도 주장이므로 근거를 요구한다 | 근거 없는 해결 방법이 다시 새어들어왔다는 뜻                    |

## 11-2. 그 결정론을 테스트로 고정했습니다 (계속)

```
$ pytest tests/
..... [100%]
32 passed, 1 warning in 0.72s
```

**test\_edge.py** 는 크래시 방지용이 아닙니다. 아래 절(11-1)의 3값 원칙이 코드에서 지켜지는지 검사하는 회귀 방어선입니다.

| 테스트                                          | 지키는 원칙                            |
|----------------------------------------------|-----------------------------------|
| test_empty_profile_is_indeterminate          | 필수 상태가 없으면 ok 가 아니라 indeterminate |
| test_null_value_is_classified_as_unknown     | null 은 충족·미충족으로 추측하지 않고 unknown   |
| test_unknown_goal_returns_404                | 없는 목표는 서버 오류(500)가 아니라 404        |
| test_numeric_string_is_classified_as_unknown | 숫자 자리에 문자열이 오면 크래시 대신 unknown     |
| test_empty_condition_tree_returns_ok         | 조건이 없는 트리도 예외 없이 판정               |

위 2,000회와 이 테스트는 목적이 다릅니다. 2,000회는 조합당 200회를 돌린 **1회성 실측**이고, pytest 의 조합당 10회는 **커밋마다 돌아가는 회귀 테스트**입니다. 전자는 규모를, 후자는 지속성을 증명합니다.



# 11-3. 그리고 화면에서 직접 확인하실 수 있습니다

측정치와 테스트는 저희가 한 말입니다. **직접 눌러 보시는 편이 빠릅니다.**

데모 화면의 「값을 바꾸면 판정이 바뀝니다」에서 사용자 상태를 바꾸면 판정이 **즉시 다시** 계산됩니다.

| 눌러 보실 것              | 그때 일어나는 일                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------|
| 해외원화결제(DCC) 차단 → 아니오 | 미충족 <b>3</b> → <b>2</b> . 판정은 여전히 blocked                                                                |
| 막힌 조건 3개를 모두 해제      | blocked → ok                                                                                             |
| 필수 조건을 → 모름          | 그 조건이 unknown 으로 이동, 판정이 확인할 수 없습니다 로 바뀝니다                                                               |
| 참고 조건을 → 모름          | unknown 으로는 하지만 판정은 가능합니다 그대로입니다. 판정은 필수 조건만 보기 때문이며, 화면이 «막는 조건은 없습니다. 다만 확인하지 못한 값 N개가 남아 있습니다» 로 밝힙니다 |

**하나만 풀어도 안 된다는** 저희 주장이 화면에서 그대로 재현됩니다. 이 계산에도 LLM 은 없습니다. 값이 같으면 몇 번을 눌러도 같은 답이 나옵니다.

**안전 장치.** 바꾼 값은 **그 요청에만** 적용되고 프로필 파일은 바뀌지 않습니다. card · account · context 안에 **이미 있는 키만** 바꿀 수 있습니다. 없는 키를 만들 수 있게 하면 조건 트리에 없는 상태를 지어내는 셈이고, 그 값이 맞는지는 아무도 보증할 수 없습니다. (tests/test\_overrides.py 가 지킵니다)

# 11-1. 판정은 3값입니다

| 값                    | 뜻                                           |
|----------------------|---------------------------------------------|
| blocked              | 차단 조건이 미충족 — 지금은 되지 않습니다                    |
| <b>indeterminate</b> | 미충족은 없지만 <b>판정에 필요한 값을 몰라</b> 된다고 말할 수 없습니다 |
| ok                   | 차단 조건을 전부 확인했고 모두 충족                        |

indeterminate 가 있는 이유 — **모르는 상태에서 "됩니다"라고 답하지 않기 위해서**입니다. 값을 모르는데 ok 를 주면 그건 추측이고, 이 시스템의 설계 원칙 위반입니다.

전부 ✓/✗ 로 채운 화면이 더 깔끔해 보이지만, **모르는 것을 안다고 표시하는 것이 거짓말**입니다. 그래서 unknown 을 화면에 그대로 노출합니다.

## 12. 해결 계획 — "하나만 풀면 되나요?"

조건을 나열해 주는 것만으로는 이 프로젝트가 시작된 경험이 해결되지 않습니다. **하나를 풀어도 여전히 막힌다는 사실을 먼저 말해야 합니다.**

판정 – 조건 11개 중 미충족 3 · 확인불가 1

|                     |                    |
|---------------------|--------------------|
| × 해외원화결제(DCC) 차단 해제 | 앱에서 가능: KB Pay     |
| × 해외거래정지 해제         | 앱에서 가능: KB Pay     |
| × IC칩 비밀번호 등록       | 앱 밖에서만: 영업점 또는 ATM |

— 하나만 풀면 되나요? —

|                 |                           |
|-----------------|---------------------------|
| 3개 중 어느 하나만 해결  | → 여전히 안 됩니다 · 필수 조건 2개 남음 |
| 앱에서 할 수 있는 것 모두 | → 여전히 안 됩니다 · 필수 조건 1개 남음 |
| 3개 모두           | → 됩니다                     |

※ 서버는 5행을 다 내지만, 결과가 같은 행은 화면에서 묶습니다.  
같은 문장을 세로로 쌓아도 정보가 늘지 않기 때문입니다.

**이 계산에도 LLM을 쓰지 않습니다.** 판정 결과를 집합 연산으로 다시 평가할 뿐이며, **없는 사용자 값을 지어내지 않습니다** — 값을 만들면 그 값이 맞는지 아무도 보증할 수 없습니다.

# 13. 측정 결과

| 지표                | 방법                                                       | 결과                                  |
|-------------------|----------------------------------------------------------|-------------------------------------|
| 조건 추출 규모          | 문서 8건 → 청크 280 → 후보 276 → 병합 16                          | 조건 12개                              |
| 검수 통과율            | 병합된 조건 중 트리에 들어간 비율                                      | 12 / 16 = 0.75                      |
| 근거 실재율            | 트리의 모든 인용이 원문에 있는가                                       | 12 / 12 = 100%                      |
| 근거 표시율            | 판정 결과에 근거가 붙는 비율                                         | 구조적 100% (스키마 필수)                   |
| 앱 안에서 해결 가능       | actionable_in_app is True                                | 해외결제 2 / 11                         |
| 앱 밖에서 해결 (근거 확인됨) | ... is False                                             | 해외결제 3 / 11                         |
| 해결 경로 미확인         | ... is None                                              | 해외결제 6 / 11                         |
| 결정론성              | 2,000회 반복 일치율 (1회성 실측)                                   | 100%                                |
| 회귀 테스트            | pytest tests/ — 결정론 1 · 엣지 6 · 상태변경 8 · 근거묶음 10 · 적용범위 7 | 32 passed                           |
| 수동 라벨 대비          | 정밀도 / 재현율                                                | eval/results/extraction_accuracy.md |

# 13-1. 수동 라벨과의 차이를 정직하게 다룹니다

---

- 자동 추출이 **라벨에 없던 유효 조건 4개**를 새로 찾았습니다 (IC칩 비밀번호 · 명의도용 차단 · 카드 잠김 · 1일 한도)
- 반대로 라벨에만 있던 1건은 **검토 결과 라벨 쪽이 틀렸습니다** — 방향을 뒤집어 만든 조건이었고, LLM 이 조건이 아니라고 판정한 것이 옳았습니다
- **라벨 파일은 고치지 않았습니다.** 자를 나중에 고치면 그 자로 켜 수치가 의미를 잃습니다. 결함은 `eval/labels/errata.yaml` 에 기록하고 보고서가 함께 출력합니다

# 14. 한계 — 숨기지 않습니다

| 한계                       | 우리의 처리                                                                                                     |                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 사용자 상태는 가상입니다            | 실제 연동에는 내부 API가 필요합니다. 이 시스템에서 가짜는 상태 하나뿐이고, 조건·근거는 전부 실제 문서에서 나옵니다                                        |                                                        |
| 판정 응답 0.056ms 는 로직 비용입니다 | 실제 서비스에는 상태 조회 시간이 더해집니다. 그래서 속도 배수를 강조하지 않습니다                                                             |                                                        |
| 약관은 개정됩니다                | 모든 조건에 수집 일자를 남기고, 트리의 유효성은 가장 넓은 근거를 따릅니다                                                                 |                                                        |
| 표에서 뽑은 인용 1건             | 셀을 `w`                                                                                                     | `로 이은 형태라 원문 HTML 검색으로는 나오지 않습니다 (각 셀은 실재). README에 명시 |
| 커버리지                     | 지금은 목표 2종입니다. 조건 트리는 목표 단위로 추가되는 구조입니다                                                                     |                                                        |
| 자연어 목표 식별은 아직 없습니다       | 목표가 2종이라 직접 선택으로 충분했고, LLM 을 데모 경로에 넣으면 실패 처리가 따라붙습니다. 15장 개발 계획 1단계입니다 — 기획안 파이프라인 그림에도 구현·계획을 구분해 표시했습니다 |                                                        |
| 수집 범위                    | robots.txt 가 막은 페이지는 자동 수집하지 않고, 사람이 공개 안내 본문만 확보해 collection_method: manual 로 기록합니다                       |                                                        |

로그인 뒤 개인화 페이지는 수집하지 않습니다. 공시·안내 페이지만 사용합니다.

# 15. 개발 계획

## 예선 이후 — 무엇을 어떤 순서로

| 단계         | 내용                              | 검증 방법                                             |
|------------|---------------------------------|---------------------------------------------------|
| 1. 목표 식별   | 자연어 질의 → goal_id (src/resolve/) | 질의 100건 라벨 대비 정확도. 못 찾으면 "아직 다루지 않는 주제" 로 정직하게 답함 |
| 2. 커버리지 확장 | 목표 2종 → 10종 (대출·카드발급·이체한도 등)    | 목표당 조건 추출 → 검수 통과율 유지 여부                          |
| 3. 상태 연동   | 가상 프로필 → 내부 API                 | 조회 실패 시 unknown 유지 (추측 금지)                        |
| 4. 개정 감시   | 문서 재수집 → 조건 변경 감지               | 트리 diff 알림. 변경된 조건은 재검수 대상                        |
| 5. 운영 지표   | 판정 후 실제 해결 여부 추적                | "지목한 원인이 맞았는가" 사후 검증                              |

## 확장이 쉬운 이유

새 목표를 넣을 때 **코드를 고치지 않습니다**. sources.yaml 에 문서를 추가하고 → 파이프라인을 돌리고 → 검수하면 트리가 생깁니다.

# 15-1. 비워 둔 칸을 어떻게 메우는가

## 지금 조건 12개 중 6개는 해결 경로가 비어 있습니다

이것은 추출 실패가 아닙니다. 문서에 없는 것입니다. 원문 전체를 다시 검색해 확인했고, 없다는 사실을 화면에 그대로 적었습니다.

| 왜 비었나                                | 다음 단계                                   | 무엇으로 확인하나                |
|--------------------------------------|-----------------------------------------|--------------------------|
| 공개문서가 조건은 말하고 해결 방법은 말하지 않는다         | 수집 범위 확장 — 앱 내 도움말 · 고객센터 안내 · 상품 약관    | 근거가 붙은 조건 수 / 여전히 빈 조건 수 |
| 문서에 없어 사람이 앱에서 직접 확인한 건이 1개 (DCC)    | 실측을 기록 양식으로 고정 (basis: measured)        | 조건마다 측정 기록 파일이 붙는가       |
| 채널은 아는데 앱 안 경로를 모른다                  | KB Pay 화면 매핑                            | 그 링크가 실제로 그 설정 화면을 여는가   |
| 조건의 적용 대상을 사람이 붙이고 있다 (applies_when) | 추출 단계로 올린다 — 인용의 «체크카드의 경우» 같은 한정어를 형식화 | 사람이 붙인 것과 자동 추출이 일치하는가   |

메우지 못하면 비워 둡니다. 6을 0으로 만드는 것이 목표가 아닙니다. 지금의 답은 빈 칸이 왜 비었는지 화면이 말하는 것이고, 다음 단계는 빈 칸을 줄이되 채운 근거를 함께 남기는 것입니다. 근거 없이 채우는 순간 이 시스템은 다른 종류의 물건이 됩니다.



# 16. 기술 실현 가능성

지금 돌아갑니다. 제3자도 돌릴 수 있습니다.

|         |                                                                                                                          |
|---------|--------------------------------------------------------------------------------------------------------------------------|
| 배포      | Vercel 서버리스 — <a href="https://kb-ai-challenge-2026.vercel.app">https://kb-ai-challenge-2026.vercel.app</a> (설치 없이 열립니다) |
| 배포본 의존성 | fastapi · pydantic · python-dotenv 3개뿐 — openai 가 없습니다                                                                   |
| 코드 규모   | 파이썬 33개 파일 / 3,533줄 (최대 파일 239줄)                                                                                         |
| 외부 의존   | 개발 전체로도 8개(requirements.txt). HTML 파서도 표준 라이브러리입니다                                                                       |
| 재현 절차   | docs/REPRODUCE.md — 새 폴더에서 처음부터 돌린 로그                                                                                    |

배포본 의존성에 openai 가 없다는 것은, 판정에 LLM 이 없다는 사실이 의존성 목록에서 그대로 보인다는 뜻입니다. 그래서 배포 환경에 넣을 API 키도 없습니다.

재현 시험이 실제 결함을 잡았습니다

pip 은 requirements.txt 를 시스템 로케일로 읽습니다. 한국어 Windows(cp949)에서 UTF-8 한글 주석 때문에 설치 자체가 실패했습니다 — 심사 환경에서 가장 흔한 조합입니다. 새 폴더에서 실제로 돌려보지 않았다면 그대로 제출했을 결함입니다.

# 16-1. 유지보수와 보안

---

## LLM이 만든 데이터를 화면에 그립니다 — 그래서 렌더링도 믿지 않습니다

판정에서 LLM을 배제한 것과 같은 이유로, **화면도 트리를 신뢰하지 않습니다.**

- 모든 출력이 이스케이프를 거칩니다. 링크는 `http(s)` 만 허용합니다 — `javascript:` 같은 스킴이 트리에 섞여 들어올 여지를 남기지 않습니다
- 값을 빠르게 여러 번 바뀌어도 **늦게 도착한 옛 응답이 최신 판정을 덮지 않습니다.** 느린 응답 1.8초를 주입해 재현한 뒤 요청 일련번호로 막았습니다
- 다시 그릴 때 **포커스·펼쳐 둔 근거·스크롤 위치를 보존합니다.** 값 하나 바꿀 때마다 근거가 접히고(6개→2개) 버튼이 튀던 것을(225~316px) 실측하고 고쳤습니다
- 상태를 색으로만 말하지 않습니다. 세 버튼은 `aria-pressed`, 판정 갱신은 `role="status"` 로 알리고, 본문 대비는 4.5:1 이상입니다
- `/api/health` 로 **배포본이 어떤 데이터를 보고 있는지** 즉시 확인할 수 있습니다

# 나중에 고칠 것을 전제로 만들었습니다

| 규칙                  | 상태                                                                                        |
|---------------------|-------------------------------------------------------------------------------------------|
| 파일 200줄 / 함수 40줄 상한 | 33개 중 31개 준수. api/main.py(239)·judge/schema.py(216) 2개가 기능 추가로 상한을 넘었고, 분리된 제출 후 과제로 남깁니다 |
| 파이프라인 단계 = 폴더 하나    | 새 단계가 생기면 폴더를 만들지, 기존 파일을 부풀리지 않습니다                                                       |
| 인터페이스 계약 C1~C4      | judge/schema.py 한 곳에 고정 — 세 사람이 병렬로 일하는 점점입니다                                             |

## 보안 — 장치를 넣고, 실제로 뚫어봅니다

- .env 는 .gitignore + **pre-commit** 훅으로 이중 차단. 훅은 감지해도 **값을 출력하지 않습니다**
- 제출 서류(성명·생년월일·연락처)도 같은 방식으로 저장소 진입을 차단합니다
- LLM 캐시에는 응답 구조체만 저장하고 요청 헤더는 남기지 않습니다

훅을 넣은 뒤 **가짜 파일로 실제 커밋을 시도**했더니, 한글 파일명이 그대로 통과했습니다. git 이 비ASCII 경로를 8진수로 이스케이프해 출력하기 때문이었습니다. 막는 장치를 넣는 것과, 실제로 막히는지 확인하는 것은 다른 일입니다.

# 17. 팀과 역할

|    | 담당                                   |
|----|--------------------------------------|
| 팀장 | 문제 정의 · 조건 추출 설계(프롬프트·어휘·검수) · 근거 검증 |
| 팀원 | 검증 스위트(결정론성·엣지) · 클린설치 재현 · 기술 문서    |

## AI 활용

코드 작성·문서 정리에 Claude 를 활용했습니다. 다만 무엇을 만들지, 어떤 조건을 트리에 넣을지, 어떤 수치를 주장할지는 사람이 판단했습니다.

이 문서의 모든 수치는 저장소의 코드와 데이터에서 직접 산출한 값입니다. 추정치는 쓰지 않았고, 측정하지 못한 것은 측정하지 못했다고 적었습니다.

# 다 됐나요?

---

조치를 하나 끝내고 재시도하며 던지는 질문이자, 앱이 끝내 대답하지 않는 질문입니다.

역대 팀들은 약관을 사람에게 쉽게 설명했습니다. 저희는 약관을 기계가 판정할 수 있게 만듭니다.

데모 — <https://kb-ai-challenge-2026.vercel.app> (설치 없이 열립니다)

저장소 · 재현 절차는 제출물에 포함되어 있습니다.

모든 조건에는 출처 URL · 원문 인용 · 수집 일자가 붙어 있습니다.